ZXSPRD AI Event Intelligence Platform

Front-End Developer Blueprint & Working Demo Technical Specification

1. Project Overview & Developer Objectives

The ZXSPRD AI Event Intelligence Platform is an enterprise-grade solution designed to manage physical event pods, live audio transcription, multi-role microphone mapping, dynamic knowledge bases, and LLM orchestration across edge devices.

This blueprint defines the architecture and implementation specifications for a fully functional, interactive front-end prototype. The UI/UX implementation must operate as a completely interactive state engine where all controls, modals, routing, state persistence, dynamic metrics, and administrative panels operate without requiring a live backend cluster.

Primary Engineering Goals

- **Full Interactive Fidelity:** Every button, modal, form input, slider, table pagination element, and routing trigger must execute state mutations in local application storage.
- **Decoupled Backend Pipeline:** The backend orchestration (Oracle Cloud Infrastructure, LLM routing, Vector Databases, Websocket telemetry streams, and Neural TTS generation) is entirely mocked using local state hooks and deterministic delay generators.
- **Unified UI State Architecture:** A single global store manages cross-application state transitions. Edits performed in the Super Admin Console instantly reflect within the Client Tenant and Edge Pod interfaces upon switching context.
- **Offline-Capable PWA:** Executable as an offline Progressive Web App for presentation reliability in low-connectivity environment scenarios.

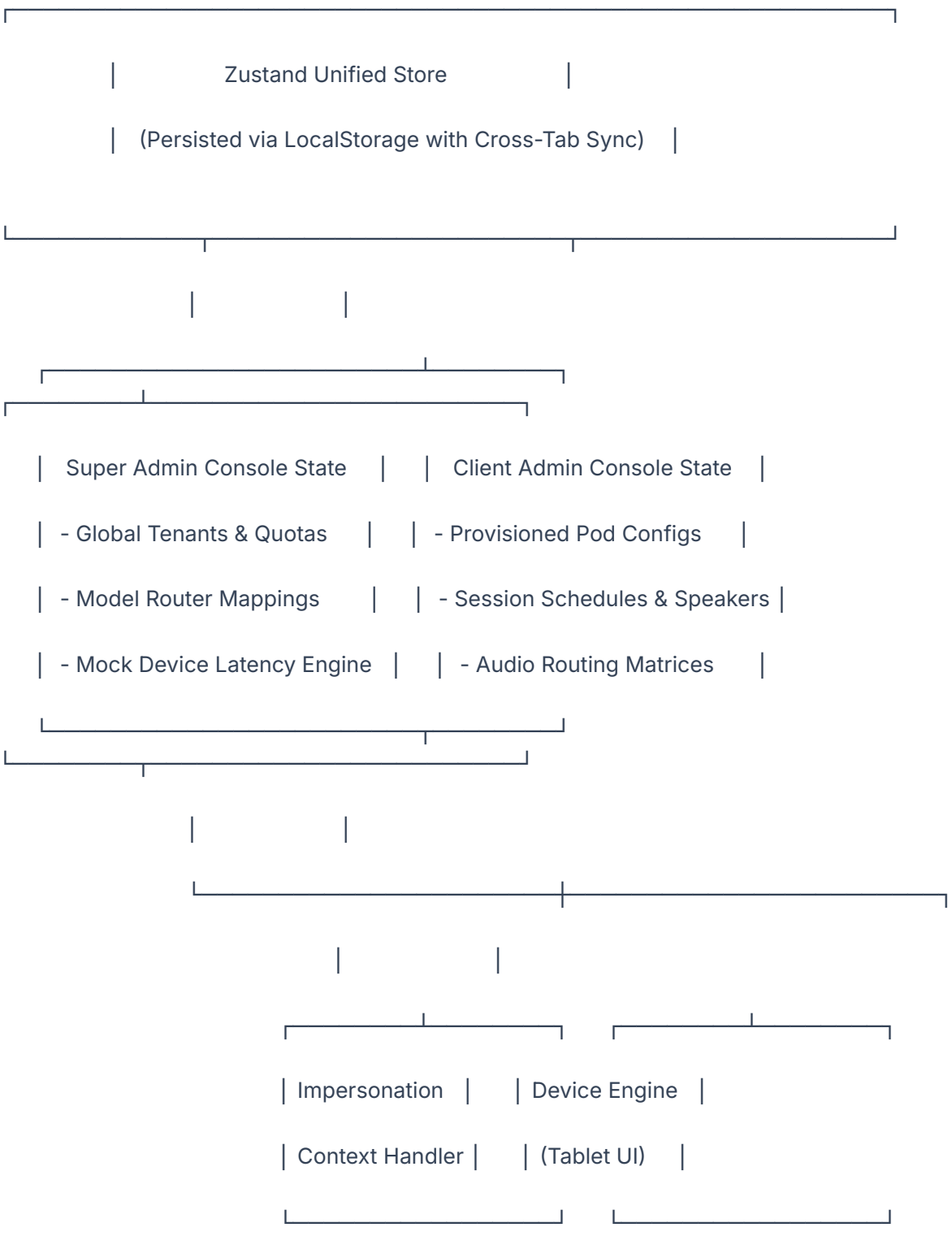
---

2. Architecture & Tech Stack Recommendations

Recommended Stack

- **Framework:** Next.js (App Router) using Client-side state hydration or React with Vite for fast build targets.
- **Styling & Components:** Tailwind CSS combined with Radix UI primitives / Shadcn UI components for accessible, accessible design system components.
- **Icons & Assets:** Lucide React for modern, standardized iconography.

- **State Engine:** Zustand with `persist` middleware (utilizing `localStorage` or `sessionStorage` fallback drivers) to maintain complex state graphs across simulated tenant role shifts.
- **Data Mocking Strategy:** MSW (Mock Service Worker) for network-level request intercept, combined with client-side reactive state generators for zero-latency UI testing.



Application Directory Structure

- `src/app/` — Application Routing & Layout Wrappers
  - `(super-admin)/` — Platform Owner Console Routes
  - `(client-admin)/` — Event Organizer Dashboard Routes
  - `(edge-device)/` — Tablet Interface & Kiosk Displays
- `src/components/` — Reusable Design System Modules
  - `ui/` — Base Radix/Shadcn Components (Dialog, Select, Table, Toast)
  - `telemetry/` — Real-Time Signal Simulators & Gauges
  - `audio/` — Visualizers, Mic Mapping Nodes, & Audio Playback Simulation
- `src/store/` — State Architecture
  - `useGlobalStore.ts` — Centralized state slice combining Tenants, Pods, Routing Rules, and Logs
- `src/mock/` — Fixture Engine
  - `telemetryWorker.ts` — Background loop updating latency/CPU statistics
  - `llmResponses.ts` — Deterministic LLM response payloads for Edge Concierge testing

---

3. Module 1: The Super Admin Console (Global View)

The Super Admin Console provides platform owners with global oversight across multi-tenant deployments, centralized AI infrastructure routing, and real-time edge device health analytics.

3.1 Tenant & Client Management Engine

A fully editable grid tracking all client instances, assigned service tiers, active pod allocation, and account health state.

- **Primary Component:** Enhanced Data Table (`src/components/admin/TenantTable.tsx`)
- **State Operations:** `addTenant()`, `updateTenant()`, `toggleTenantStatus()`

Field Name	Type	UI Control / Display	Action Behavior
Tenant ID	String	Monospace Tag	Auto-generated UUID (`tnt_9941`)
Organization Name	String	Editable Text Cell	Updates inline or via Modal
Service Tier	Enum	Badge (`Enterprise`, `Pro`, `Custom`)	Alters max pod quota limits
Pod Allocation	Fraction	Progress Bar + Text (`8 / 10`)	Validates quota prior to creation

Field Name	Type	UI Control / Display	Action Behavior
LLM Provider Target	String	Status Chip (`OCI Llama3`, `OpenAI`)	Driven by Global Model Router
Status	Enum	Switch Toggle (`Active` / `Suspended`)	Disables Client access instantly
Actions	Trigger	Dropdown Menu	Includes "Impersonate", "Edit", "Suspend"

```
// Mock Interface Specification: Tenant Object

interface Tenant {

 id: string;

 name: string;

 tier: 'Standard' | 'Pro' | 'Enterprise';

 maxPods: number;

 assignedPods: number;

 modelTarget: string;

 ttsProvider: string;

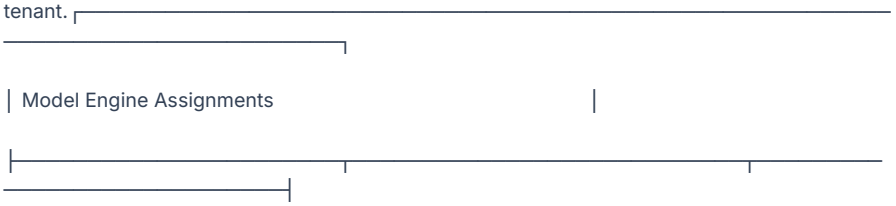
 status: 'Active' | 'Suspended';

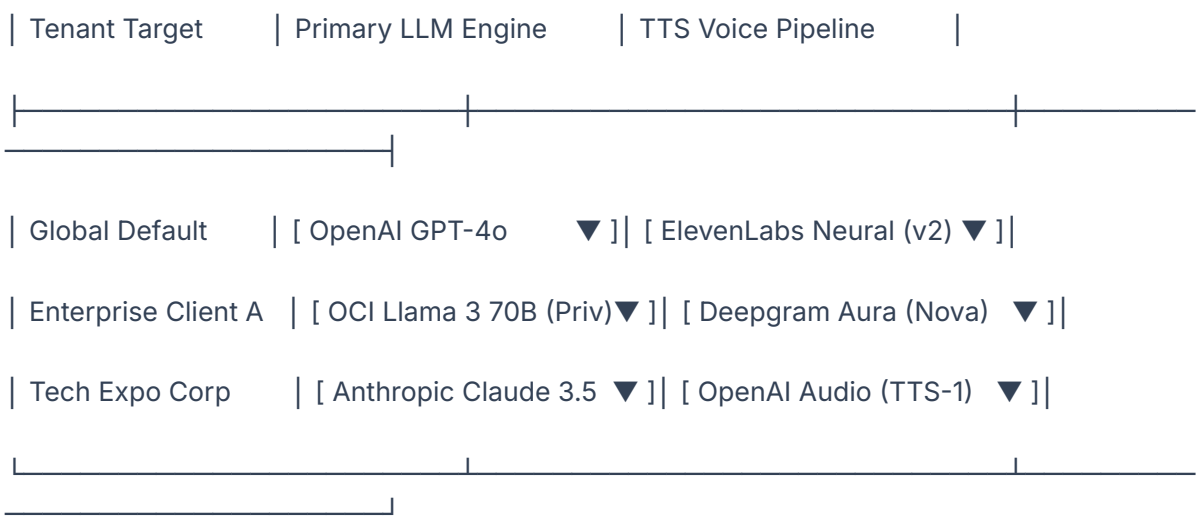
 createdDate: string;

}
```

3.2 Global Model Router & LLM Engine Configurator

Enables global assignment of LLM instances, fallback routing policies, temperature parameters, and Neural TTS voice engines per tenant.





- **Dynamic Features:**
  - **Fallback Toggle:** Enable automatic failover to public cloud API if local edge OCI inference latency exceeds 800ms.
  - **Temperature Slider:** Continuous range from `0.0` (Precise/Factual) to `1.0` (Creative). Value updates in Zustand store and immediately changes the simulated latency and response complexity on the Edge Pod.

3.3 Live Device Telemetry Engine

A mock background thread drives dynamic telemetry metrics to simulate real-world hardware monitoring across physical event pods.// Telemetry Loop Implementation Spec

```
export const startTelemetrySimulation = (updateDeviceStore: Function) => {

  setInterval(() => {

    const updatedMetrics = {

      pingLatency: Math.floor(Math.random() * (120 - 18 + 1)) + 18, // 18ms to 120ms

      cpuUtilization: Math.floor(Math.random() * (88 - 12 + 1)) + 12, // 12% to 88%

      memoryUsage: (Math.random() * (7.2 - 3.1) + 3.1).toFixed(1), // 3.1 GB to 7.2 GB

      batteryStatus: Math.min(100, Math.floor(Math.random() * 10) + 90),

      status: Math.random() > 0.03 ? 'Online' : 'Degraded'

    };

    updateDeviceStore(updatedMetrics);
```

```
  }, 2500);

};
```

3.4 Context-Switching (Client Impersonation)

A persistent navigation control bar allows the Super Admin to select any active tenant from a header dropdown. Selecting a tenant sets `activeTenantId` in the root state context, instantaneously altering all navigation routing to reflect the perspective of that specific Client Admin.

---

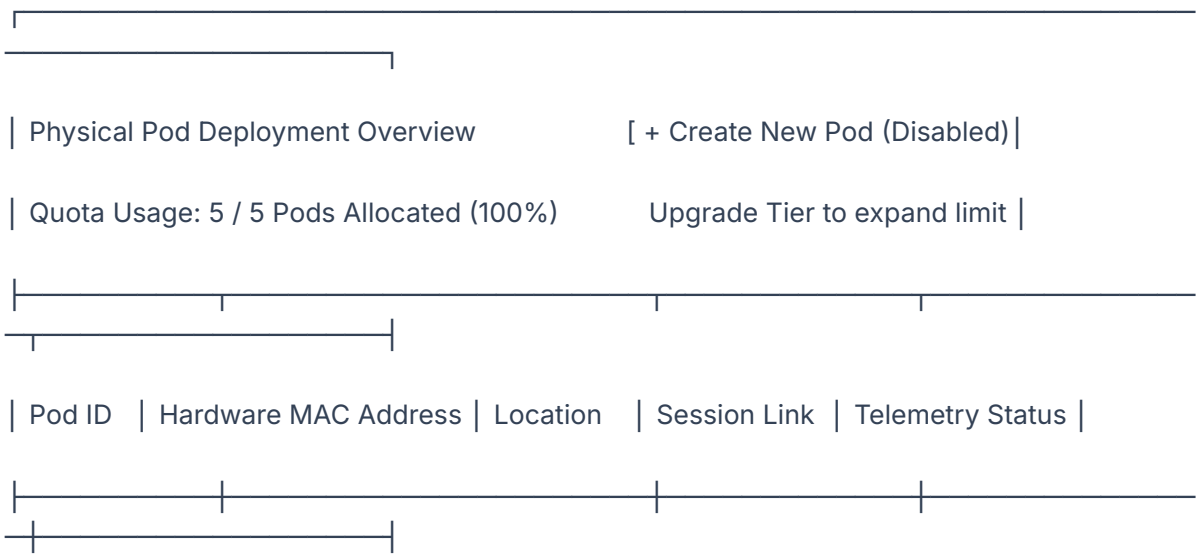
4. Module 2: The Client Admin Console (Tenant View)

The event organizer console controls physical venue layout configurations, agenda scheduling, audio channel mapping, and custom knowledge ingestion.

4.1 Pod & Table Provisioning System

Interface for registering physical edge hardware units assigned to specific venue spaces (e.g., "Pod 01 - VIP Lounge").

- **Quota Enforcement Logic:**
  - The "Create New Pod" button reads `assignedPods` and `maxPods` from the active Tenant context.
  - If `assignedPods >= maxPods`, the button enters a `disabled` visual state accompanied by an explanatory tool-tip: *"Tenant Quota Reached. Upgrade tier in Super Admin Console."*
  - Deleting a pod decrements the count instantly and re-enables creation.



| POD-01 | AC:DE:48:00:11:22 | Hall A - T1 | Keynote Intro | 24ms / CPU 14% |

| POD-02 | AC:DE:48:00:11:23 | Hall A - T2 | VIP Roundtable | 31ms / CPU 22% |

4.2 Agenda & Meeting Schedule Builder

A interactive timeline and modal system to create, sequence, and modify event sessions.interface EventSession {

id: string;

podId: string;

title: string;

startTime: string; // "10:00 AM"

endTime: string; // "10:45 AM"

speakerHost: string;

speakerGuest: string;

status: 'Scheduled' | 'Live' | 'Completed';

}

- **Interactive Features:** Drag-and-drop or step-modal session builder, automatic time collision detection, and "Trigger Session Start" button that immediately updates the active Edge Device display.

4.3 Multi-Mic Role Mapping Architecture

A core innovation of the ZXSPRD platform is real-time dynamic mic channel mapping to identify distinct event participants. [Physical Hardware Interface]

Microphone Hardware 1 ──┐

├─▶ [Visual Routing Node] ──▶ Host Channel (Speaker 1)

Microphone Hardware 2 ──┘ ──▶ Guest Channel (Speaker 2)

- **UI Implementation:** A drag-and-drop connector grid or high-contrast drop-down matrix.

- **Control Binding:**
    - Assigning **XLR/USB Channel 1** to `Host (Dr. Aris Thorne)` binds incoming simulated voice activity on Channel 1 to Dr. Thorne's transcript stream.
    - Assigning **XLR/USB Channel 2** to `Guest (Elena Rostova)` binds Channel 2 to the guest visual card.
    - Includes audio gain meters (animated visualizers running via SVG/CSS keyframes).

4.4 Dynamic Dynamic Knowledge Base & Override Console

- **Mock Document Dropzone:**
    - Drag-and-drop interface for ingestion of PDF, DOCX, or TXT venue guides.
    - Dropping a file initiates a simulated multi-stage ingestion pipeline progress bar:
        i. *Uploading Document Stream...* (0 - 30%)
        ii. *Parsing Text & Layout Visuals...* (30 - 65%)
        iii. *Generating Vector Embeddings (1536 dim)...* (65 - 95%)
        iv. *Vector Index Ready* (100%)
- **Instant Override Announcement Engine:**
    - A persistent high-priority text field.
    - Inputting text (e.g., *"Emergency Notice: Keynote in Hall B delayed by 15 minutes."*) and clicking **Broadcast Now** immediately pushes an alert banner to all active Edge Pods.

---

5. Module 3: Edge Device UI (The Tablet Experience)

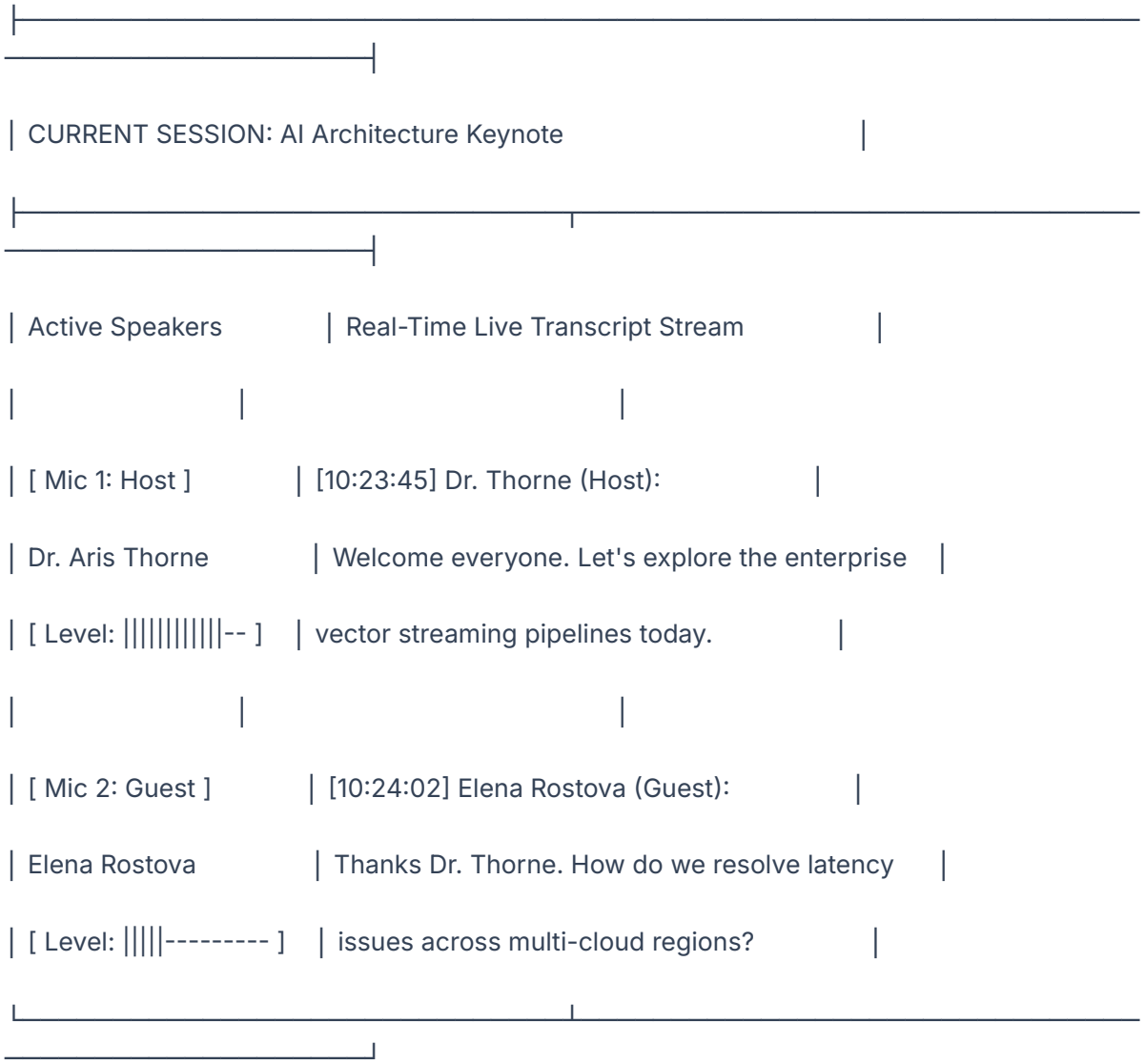
Designed for physical Android/iOS/Web tablet hardware mounted directly on event tables or pod stands. Must render flawlessly at $16:10$ and $16:9$ aspect ratios in kiosk mode.

5.1 Idle Standby Screen

- High-contrast design featuring event logo, pod identification, current time, and network connection status.
- Displays dynamic call-to-action text: *"Touch screen or pick up microphone to activate concierge."*
- Integrates subtle, ambient micro-animations (CSS gradients) to signal device availability without display burn-in risks.

5.2 Active Session View

When a session is marked active in the Client Admin Console, the tablet interface updates automatically via unified state
hooks. ┌──────────────



5.3 Voice Concierge Interface (Mock Pipeline)

An interactive AI assistant simulation configured with deterministic response trees.

- **Interaction Lifecycle Execution:**
    a. **Idle State:** User clicks "Tap to Speak" button.
    b. **Listening State:** Button expands into an animated audio waveform component. A 3-second timer initiates.
    c. **Transcribing State:** Displays hardcoded user transcript string word-by-word: *"Where is the VIP networking lounge located?"*
    d. **Processing State:** Displays animated loader: *"Searching Knowledge Base Vector Index..."*
    e. **Response State:**
        - Displays textual response card: *"The VIP Networking Lounge is located on Level 2, directly across from Escalator B."*

- Triggers HTML5 Web Speech API (`window.speechSynthesis.speak()`) to read the response out loud in a clear neural voice.

```
// Voice Concierge Execution Script Spec

export const simulateConciergeInteraction = (

  setAppState: Function,

  userQuery: string = "Where is the VIP networking lounge located?",

  aiResponse: string = "The VIP Networking Lounge is located on Level 2, directly across from Escalator B."

) => {

  setAppState({ conciergeState: 'LISTENING' });



  setTimeout(() => {

    setAppState({ conciergeState: 'PROCESSING', transcript: userQuery });

  }, 3000);

  setTimeout(() => {

    setAppState({ conciergeState: 'RESPONDING', response: aiResponse });



    if ('speechSynthesis' in window) {

      const utterance = new SpeechSynthesisUtterance(aiResponse);

      utterance.rate = 1.0;

      utterance.pitch = 1.0;

      window.speechSynthesis.speak(utterance);

    }

  }, 4800);
```

};

---

6. Demo Execution & Storyboarding (The "Golden Path")

Follow this sequential click-path during investor previews or executive demonstrations to highlight platform capability and operational integration. [STEP 1] [STEP 2] [STEP 3] [STEP 4]

Super Admin Tenant Switch Client Admin Edge Device

Configure Model Engine —▶ Impersonate Client —▶ Update Mic Mapping —▶ Trigger Concierge

& Set Pod Quota "Enterprise Client A" & Push Broadcast Announcement Voice Cycle

Step-by-Step Demonstration Sequence

Step 1: Global Platform Management (Super Admin Console)

1. Open the application. Select the **Super Admin Console** view from the top navigation context switcher.
2. Navigate to **Global Model Router**. Change "Enterprise Client A" LLM assignment from `OpenAI GPT-4o` to `Private OCI Llama 3 70B`. Adjust the Temperature Slider to `0.2`.
3. Highlight the **Live Telemetry Engine**. Point out the ping fluctuations and memory metrics updating dynamically.
4. Click the **Impersonate Tenant** button next to "Enterprise Client A".

Step 2: Event Pod Configuration (Client Admin Console)

1. Observe that the navigation context automatically updates to "Enterprise Client A" view.
2. Attempt to click **Create New Pod**. Note how the UI enforces the assigned max quota limit ($5/5$).
3. Navigate to **Multi-Mic Role Mapping**. Change **Mic Channel 1** drop-down to `Keynote Host` and **Mic Channel 2** to `Panelist Guest`.
4. Go to **Dynamic FAQ / Ingestion**. Drag a sample PDF into the upload dropzone and watch the vector processing progress bar complete.
5. In the **Instant Override Console**, type: `"Session starting in 5 minutes"` and click **Broadcast Now**.

Step 3: Edge Tablet Interaction (Edge Device Kiosk View)

1. Switch route/view to **Edge Device UI (Pod 01 Tablet)**.
2. Notice the instant top alert banner displaying: *"Session starting in 5 minutes"* (pushed directly from the Client Admin override).

3. Observe the **Live Session Monitor** displaying the mapped speakers (Host and Guest) updated from Step 2.
4. Click the **Tap to Speak (Voice Concierge)** button.
5. Observe the state transitions: **Listening** $\rightarrow$ **Transcribing** $\rightarrow$ **Vector Processing** $\rightarrow$ **Synthesized Audio Response Output**.

---

7. Core Component State Verification Matrix

Use this matrix to verify front-end functionality before demo presentation.

Feature Area	Interactive Target	Expected Frontend Mutation State	Persistence Scope
Tenant Control	Suspend Toggle	Toggles client state; locks out Client Admin access	LocalStorage / Global Store
Model Router	Provider Dropdown	Updates target engine label across all linked Pod displays	LocalStorage / Global Store
Pod Allocation	Delete Pod	Decrements quota counter; enables "Create Pod" button	Session State
Mic Mapping	Channel Dropdown	Re-binds speaker labels in Live Transcript stream	LocalStorage / Global Store
Knowledge Base	Ingest File Drop	Triggers linear progress loader; appends file to index list	LocalStorage / Global Store
System Override	Broadcast Button	Instantly renders flash banner across all tablet views	Global Event Bus / Store
Voice Concierge	Tap to Speak	Cycles through audio visualization, text typing, TTS audio	Local View State

---

8. CSS & UI Theme Standards

To ensure executive-level visual polish, apply the following design parameters throughout the Tailwind component system:

- **Primary Palette:** Deep Navy (`#0F172A`), Slate Gray (`#475569`), Soft Background Accent (`#F8FAFC`).

- **Status Colors:** Success Green (`#10B981`), Warning Gold (`#F59E0B`), System Pulse Blue (`#3B82F6`).
- **Typography Scale:**
    - Titles: `Inter`, 22pt Bold, Line-Height 1.0.
    - Headings: `Inter`, 12pt - 14pt Semi-Bold.
    - Body Text: `Inter`, 10pt Regular, Line-Height 1.25.
    - Monospace / Telemetry: `JetBrains Mono` or `ui-monospace`, 9pt.
- **Component Styling:** Subdued border contrast (`#E2E8F0`), zero-width vertical table borders, rounded card edges (`rounded-xl`), and smooth background transitions (`transition-all duration-200`).